MÁSTER FP IA Y BIG DATA · MÓDULO 5073 · BLOQUE 4

Sesión 1 de 6

Integración con LLMs

APIs de OpenAI, Gemini y Claude

Cámara de Madrid · Curso 2026-2027 · Martes 18:00–21:30

EL BLOQUE

Seis sesiones, un mismo asistente

#	Sesión	Qué construimos	Entrega
1	Integración con LLMs	El mismo encargo a tres proveedores; interfaz común	E1 · 10 %
2	Salidas estructuradas y herramientas	Clasificador de correos; asistente con herramientas	E2 · 15 %
3	Agentes: bucle y razonamiento	Agente con aprobación humana; versión LangChain	E3 · 15 %
4	Memoria y contexto	Compactación; hilos y preferencias por cliente	E4 · 15 %
5	MCP: protocolo y servidor	Servidor MCP del caso; Inspector; hosts	E5 · 15 %
6	MCP: clientes y seguridad	Agente MCP por HTTP; red teaming; integrador	E6 · 30 %

EL CASO

Tostadero Retiro S.L.

Distribuidora madrileña de café de especialidad, maquinaria y consumibles para hostelería.

Bares, restaurantes, cafeterías y hoteles escriben para **hacer pedidos**, **comunicar averías** de sus máquinas o **preguntar**.

La empresa quiere un asistente que entienda esos correos, consulte sus datos y actúe **con supervisión humana**.

TOSTADERO.DB · SQLITE

```
clientes
productos
pedidos
lineas_pedido
incidencias
```

ARQUITECTURA OBJETIVO

Lo que tendréis al final del bloque



OBJETIVOS DE HOY

Al terminar la sesión serás capaz de…

1. Llamar desde Python a las APIs de OpenAI, Gemini y Claude, y leer la respuesta y el consumo de tokens.
2. Explicar qué es un token, qué es la ventana de contexto y por qué la API no guarda estado.
3. Estimar el coste y la latencia de una llamada y elegir modelo con criterio.
4. Tratar los errores habituales (credenciales, límites de uso, sobrecarga) y proteger las claves.
5. Encapsular varios proveedores tras una interfaz común (patrón Adapter).

AGENDA

Tres horas, dos teorías y dos prácticas

Hora	Bloque	Formato
18:00	Presentación del bloque, del caso y de la arquitectura	Charla
18:10	Teoría 1–2: anatomía de una llamada y las tres APIs	Teoría
18:35	P1 entorno · P2 el mismo encargo a tres proveedores	Práctica
19:35	Descanso	—
20:05	Teoría 3–7: parámetros, coste, errores, seguridad, local	Teoría
20:25	P3 interfaz común (patrón Adapter)	Práctica
20:55	P4 comparativa de proveedores	Práctica
21:20	Puesta en común y preguntas de control	Debate

ANTES DE EMPEZAR · ASISTENTE PARA ESTUDIAR

Gemini de pago: elige una de las dos

Escanéame

	Google AI Plus	Google AI Pro
Precio	Gratis 12 meses, luego 4,99 €/mes	5,49 €/mes (75 % menos que 21,99 €)
Incluye	Límites de uso 2× más altos que la versión gratuita · 400 GB	Los modelos Pro de Gemini y las funciones avanzadas · 5 TB
Requisitos	Universitario, 18 años o más · cuenta personal de Google · verificación con SheerID · método de pago · pedirla antes del 31/12/2026	Cuenta personal de Google · método de pago · condiciones de la oferta en la web de Google
Ojo	Se renueva sola: cancélala antes de que acabe el año si no quieres pagar	Descuento de hasta 4 años: revisa precio y renovación antes de pagar
Elígela si…	Quieres probar sin pagar nada	La vas a usar a diario y quieres el mejor modelo

ANTES DE EMPEZAR · SALDO PARA LAS PRÁCTICAS

Una clave de OpenRouter con 5–10 €

Recarga recomendada

5–10 €

Pasar todo el kit del bloque cuesta **≈ 0,15 $**. El resto queda para tus pruebas.

Pago por uso, sin suscripción. **Una sola clave** para OpenAI, Anthropic y Google.

1. Crea tu cuenta en **openrouter.ai**.
2. **Credits** → añade 5–10 € con tu tarjeta.
3. **Keys** → crea una clave y ponle un límite de gasto.
4. Pégala en tu .env, en **OPENROUTER_API_KEY**.

¿Sin saldo? Los modelos **:free** funcionan, con límites de uso y sin datos reales.

ANTES DE EMPEZAR · MATERIAL Y ENTREGAS

Todo el bloque, en GitHub

Material: el repo del bloque

github.com/JavierDomi/master-ia-bloque4

Una carpeta por sesión y, dentro, una por práctica con su enunciado y su código. Todo el código va a bloque4.

Entregas: tu repo privado

1. Crea un repo **privado** llamado **master-ia-bloque4-apellido-nombre**, en minúsculas y sin tildes ni eñes.
2. Settings › Collaborators: invita a **JavierDomi**.
3. Una carpeta por entregable: **e1/**, **e2/**… En e1/: llm_client.py, comparativa.csv y analisis.md.
4. Nunca subas tu **.env**: añádelo a .gitignore antes del primer commit.

TEORÍA · 18:10 · APARTADOS 1 Y 2

Anatomía de una llamada y las tres APIs

Qué viaja por la red cuando «llamas a un LLM»

Una petición HTTPS con un cuerpo JSON

Petición · Responses API de OpenAI

```
{
  "model": "gpt-5.4-mini",
  "instructions": "Eres el asistente
    de Tostadero Retiro...",
  "input": "La cafetera vuelve a
    perder presión y mañana
    tenemos el pedido 1004..."
}
```

Respuesta (abreviada)

```
{
  "id": "resp_...",
  "output": [{"role": "assistant",
    "content": [{"type": "output_text",
      "text": "Es un correo mixto..."}]}],
  "usage": {"input_tokens": 78,
            "output_tokens": 61}
}
```

Es una API REST más: modelo + instrucciones + mensajes + parámetros → texto generado + **consumo de tokens**.

1 · QUÉ OCURRE AL «LLAMAR A UN LLM»

Tres peculiaridades que condicionan el diseño

No guarda estado

El modelo no recuerda nada entre llamadas. Si no reenvías la primera pregunta, la conversación no existe. La memoria es cosa de tu aplicación (sesión 4).

Se mide en tokens

Un token es un fragmento de palabra; en español, una palabra ≈ algo más de un token. Limitan cuánto cabe (ventana de contexto) y fijan el coste.

No es determinista

La misma petición puede dar respuestas distintas. Valida la salida y no dependas de una redacción concreta.

Los mensajes tienen roles

`system`	Las instrucciones: quién es el asistente, sus normas y el formato de respuesta.
`user`	Lo que escribe el usuario: en nuestro caso, el correo del cliente.
`assistant`	Las respuestas anteriores del modelo, que reenvías para «recordar» la conversación.

Mismos conceptos, otros nombres

	OpenAI	Anthropic Claude	Google Gemini
API recomendada	Responses API	Messages API	Interactions API
Paquete Python	openai	anthropic	google-genai
Llamada	client.responses.create()	client.messages.create()	client.interactions.create()
Instrucciones	instructions=	system=	system_instruction=
Entrada	input=	messages= y max_tokens obligatorio	input=
Texto	r.output_text	bloques de r.content de tipo text	r.output_text
Tokens	usage.input_tokens, output_tokens	usage.input_tokens, output_tokens	usage.total_input_tokens, total_output_tokens
Estado en servidor	Opcional: previous_response_id	No: reenvías el historial	Opcional: previous_interaction_id

Al día: octubre de 2026

OpenAI

Responses API es la recomendada. La Assistants API se retiró el 26 de agosto de 2026. Chat Completions sigue disponible.

Google

Interactions API por defecto desde junio de 2026. SDK correcto: google-genai, no el antiguo google-generativeai.

Compatibles con OpenAI

OpenRouter, Ollama, vLLM y la propia Gemini imitan Chat Completions; OpenRouter acepta además Responses y Messages. Los SDK leen OPENAI_BASE_URL y ANTHROPIC_BASE_URL: así el kit va por OpenRouter sin cambios.

Nombres de modelo

Cambian cada pocos meses. Nuestro código los lee de una variable de entorno (.env), nunca escritos en el código.

PRÁCTICA · 18:35 · 60 MIN

P1 entorno · P2 tres proveedores

ENUNCIADO Y CÓDIGO · S1-LLM-APIS/	SOLO EJECUTAS	EJECUTA
P1-entorno/ P2-el-mismo-encargo-a-tres-proveedores/	s1_hola_tres.py	python s1_hola_tres.py

P1 · Entorno 25’

```
# en tu carpeta bloque4 (sesión 0)
source .venv/bin/activate
pip install -r requirements.txt
# .env: pega tu clave, una vez

python s1_hola_tres.py
```

Copia en bloque4 los ficheros de s1-llm-apis/ y de cada práctica. En .env solo pegas tu clave.

P2 · Ejecutadlo dos veces 35’

1. ¿Qué proveedor responde antes? ¿Se mantiene entre ejecuciones?
2. Mismo texto: ¿por qué no coinciden los tokens de entrada?
3. El correo es mixto (incidencia y pedido): ¿lo clasifican bien los tres?
4. ¿Cambia la redacción entre la primera y la segunda ejecución?

Instalación del kit y .env


```
# En tu carpeta bloque4 (la de la sesión 0), con el kit de la sesión 1 descargado del repo
source .venv/bin/activate        # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt  # por si hay dependencias nuevas
# ¿Sin puesta a punto? Guía de la sesión 0: .venv, .env y python crear_bd.py

# .env: solo cambias la primera línea; los modelos ya vienen puestos
OPENROUTER_API_KEY=sk-or-...                # tu clave, SOLO aquí
OPENAI_BASE_URL=https://openrouter.ai/api/v1
OPENAI_API_KEY=${OPENROUTER_API_KEY}
OPENAI_MODEL=openai/gpt-5.4-mini            # SDK de OpenAI · Responses API
ANTHROPIC_BASE_URL=https://openrouter.ai/api
ANTHROPIC_API_KEY=${OPENROUTER_API_KEY}
ANTHROPIC_MODEL=anthropic/claude-haiku-4.5  # SDK de Anthropic · Messages API
GEMINI_MODEL=google/gemini-3.8-flash        # SDK de OpenAI · Chat Completions
GEMINI_API_KEY=                             # opcional: clave gratuita de AI Studio → API nativa
```

P2 · S1_HOLA_TRES.PY (SIMPLIFICADO)

Las tres llamadas, lado a lado

OpenAI · Responses

```
c = openai.OpenAI()
r = c.responses.create(
  model=modelo,
  instructions=SISTEMA,
  input=PETICION)

texto = r.output_text
```

Claude · Messages

```
c = anthropic.Anthropic()
r = c.messages.create(
  model=modelo,
  max_tokens=512,
  system=SISTEMA,
  messages=[{
    "role": "user",
    "content": PETICION}])

texto = "".join(b.text
  for b in r.content
  if b.type == "text")
```

Gemini · Chat Completions

```
c = openai.OpenAI(
  base_url=OPENROUTER)
r = c.chat.completions.create(
  model=modelo,
  messages=[
    {"role": "system",
     "content": SISTEMA},
    {"role": "user",
     "content": PETICION}])

texto = (r.choices[0]
  .message.content)
```

Con GEMINI_API_KEY: interactions.create() nativa

s1_hola_tres.py


```
"""Sesión 1. El mismo encargo a tres proveedores: respuesta, tokens y latencia."""
import os
import time

import anthropic
import openai
from dotenv import load_dotenv
from google import genai

load_dotenv()

OPENROUTER = "https://openrouter.ai/api/v1"
SISTEMA = ("Eres el asistente de atención al cliente de Tostadero Retiro S.L. "
           "Responde en español y en dos frases como máximo.")
CORREO = ("Hola, soy Marta del Bar La Esquina. La cafetera vuelve a perder presión y mañana "
          "tenemos el pedido 1004 pendiente. ¿Me decís algo? Gracias")
PETICION = ("Clasifica este correo como pedido, incidencia, consulta o mixto y redacta "
            f"una respuesta breve:\n\n{CORREO}")
```

s1_hola_tres.py


```
def con_openai() -> tuple[str, int, int]:
    cliente = openai.OpenAI() # lee OPENAI_API_KEY y OPENAI_BASE_URL del entorno
    r = cliente.responses.create(
        model=os.environ["OPENAI_MODEL"], instructions=SISTEMA, input=PETICION
    )
    return r.output_text, r.usage.input_tokens, r.usage.output_tokens


def con_claude() -> tuple[str, int, int]:
    cliente = anthropic.Anthropic() # lee ANTHROPIC_API_KEY y ANTHROPIC_BASE_URL
    r = cliente.messages.create(
        model=os.environ["ANTHROPIC_MODEL"],
        max_tokens=512, # obligatorio en la API de Claude
        system=SISTEMA,
        messages=[{"role": "user", "content": PETICION}],
    )
```

s1_hola_tres.py


```
    texto = "".join(b.text for b in r.content if b.type == "text")
    return texto, r.usage.input_tokens, r.usage.output_tokens


def con_gemini() -> tuple[str, int, int]:
    if os.getenv("GEMINI_API_KEY"): # opcional: API nativa de Gemini (Interactions API)
        cliente = genai.Client() # lee GEMINI_API_KEY
        r = cliente.interactions.create(
            model=os.environ["GEMINI_MODEL"].removeprefix("google/"),
            system_instruction=SISTEMA, input=PETICION,
        )
        return r.output_text, r.usage.total_input_tokens, r.usage.total_output_tokens
    # Por defecto, vía OpenRouter con Chat Completions: el formato que imitan casi todos
    cliente = openai.OpenAI(base_url=OPENROUTER, api_key=os.environ["OPENROUTER_API_KEY"])
    r = cliente.chat.completions.create(
        model=os.environ["GEMINI_MODEL"],
        messages=[{"role": "system", "content": SISTEMA}, {"role": "user", "content": PETICION}],
    )
```

s1_hola_tres.py


```
    return r.choices[0].message.content, r.usage.prompt_tokens, r.usage.completion_tokens


if __name__ == "__main__":
    for nombre, llamar in [("OpenAI", con_openai), ("Claude", con_claude), ("Gemini", con_gemini)]:
        inicio = time.perf_counter()
        texto, t_in, t_out = llamar()
        print(f"\n=== {nombre} | {time.perf_counter() - inicio:.2f} s | "
              f"{t_in} tokens entrada | {t_out} tokens salida\n{texto}")
```

19:35 · 30 MINUTOS

Descanso

A las 20:05, teoría: parámetros, coste, errores, seguridad y modelos locales.

TEORÍA · 20:05 · APARTADOS 3 A 7

Parámetros, coste, errores, seguridad y modelos locales

Lo que separa un script de una integración seria

3 · PARÁMETROS DE GENERACIÓN

Tres mandos que cambian la respuesta

Longitud máxima

Obligatoria en Claude (max_tokens), opcional en los demás.

Corta la respuesta, no la resume.

Temperatura

Baja para extraer y clasificar; más alta para redactar con variedad.

Algunos modelos de razonamiento no la admiten.

Razonamiento

El modelo «piensa» antes de responder. Más esfuerzo: mejor en problemas complejos, más latencia y coste.

Esos tokens se facturan como salida.

```
reasoning (OpenAI)
thinking (Claude)
thinking_level (Gemini)
```

¿Cuánto cuestan 1.000 correos?

1.000 correos × 300 tokens de entrada
y 150 tokens de salida

```
entrada 300.000 × 0,40 $/M = 0,12 $
salida  150.000 × 1,60 $/M = 0,24 $
```

0,36 $

Precios hipotéticos por millón de tokens

Real · gpt-5.4-mini: 0,75 / 4,50 $/M → 0,90 $

Empieza por el modelo pequeño y sube de gama solo si la evaluación lo justifica.

La salida es más cara: pide respuestas concisas y acota la longitud.

Latencia percibida = tiempo hasta el primer token. El streaming la reduce sin cambiar el coste.

Prefijo estable (instrucciones y herramientas al principio) → caché de prompts: más barato y rápido.

5 · ERRORES Y ROBUSTEZ

No todos los errores se reintentan

Error	Significado	Qué hacer
401 / 403	Clave inválida o sin permisos	Revisar .env. No reintentar.
400	Petición mal formada	Corregir el código. No reintentar.
429	Límite de uso superado	Reintentar con espera exponencial y bajar el ritmo.
500, 503 (529 en Claude)	Fallo o sobrecarga del proveedor	Reintentar con espera; si persiste, degradar o cambiar de proveedor.
Tiempo agotado	Respuesta larga o red lenta	Reintentar una vez, usar streaming y avisar al usuario.

Tres reglas que no se negocian



Claves fuera del código

Variables de entorno o un .env incluido en .gitignore. Si una clave se filtra, se revoca de inmediato. Al terminar el curso, revoca la de OpenRouter.



Nunca desde el navegador

El navegador habla con tu backend; el backend llama al proveedor, controla el uso y registra el consumo.



RGPD antes de enviar datos

Base jurídica, minimización, contrato de encargado del tratamiento y retención del proveedor. Las prácticas no usan datos personales reales.

Ollama: el mismo código, en tu equipo

Ejecuta modelos abiertos (Llama, Qwen, Mistral, Gemma…) y expone una API **compatible con Chat Completions** de OpenAI.

```
cliente = openai.OpenAI(
  base_url="http://localhost:11434/v1",
  api_key="ollama")
```

A favor

Los datos no salen de la organización. Sin coste por token. Plan B si no hay red o claves.

En contra

Los modelos pequeños razonan peor y usan herramientas con menos fiabilidad. Conviene verlos fallar.

ANTES DE P3 · PATRÓN DE DISEÑO

Patrón Adapter: un enchufe común

SIN ADAPTADOR

```
if proveedor == "openai":
    r = cli.responses.create(...)
    texto = r.output_text
elif proveedor == "claude":
    r = cli.messages.create(...)
    texto = r.content[0].text
# …repetido allí donde llames al modelo
```

CON ADAPTADOR

```
llm = LLM(proveedor)
r = llm.responder(sistema, prompt)
texto = r.texto

# el mismo código para OpenAI,
# Claude, Gemini u Ollama
```

1 · Interfaz común

responder() → Respuesta: lo único que ve tu código.

2 · Adaptador

LLM traduce la llamada y normaliza la respuesta.

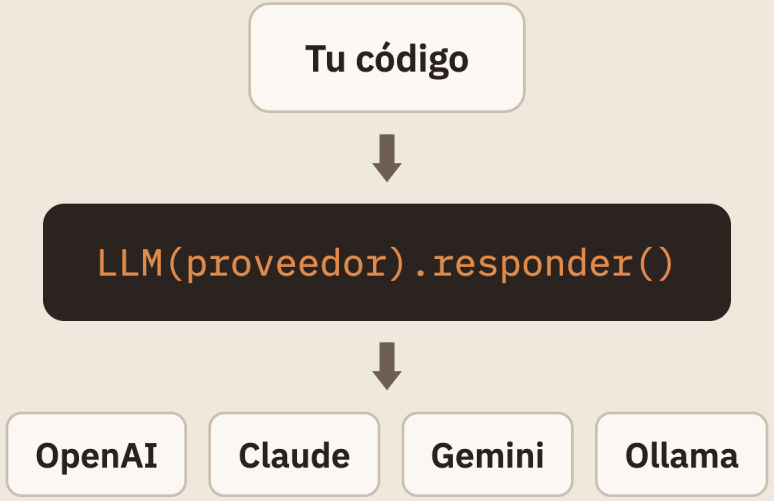
3 · Adaptados

Los SDK de cada proveedor, cada uno a su manera.

PRÁCTICA · 20:25 · 30 MIN

P3 · Una interfaz común (patrón Adapter)

ENUNCIADO Y CÓDIGO · S1-LLM-APIS/	PROGRAMAS (TODO)	EJECUTA
`P3-una-interfaz-comun/`	`llm_client.py`	`python llm_client.py`



Siempre devuelve una **Respuesta**: texto, tokens, segundos, modelo y coste

Tareas

1. Completa las ramas responses (OpenAI) y messages (Claude): busca los TODO. Modelo: la rama chat; llamadas: s1_hola_tres.py.
2. Rellena PRECIOS con openrouter.ai/models y comprueba que coste_usd da una cifra.
3. Añade max_tokens y tradúcelo al parámetro de cada proveedor.
4. Reto: responder_stream(), un generador de trozos de texto (mira s1_streaming.py).

P3 · LLM_CLIENT.PY

Lo esencial del adaptador

```
@dataclass
class Respuesta:
    texto: str
    tokens_entrada: int
    tokens_salida: int
    segundos: float
    modelo: str

class LLM:
    def responder(self, sistema, prompt,
                  temperatura=None) -> Respuesta:
        if self.formato == "responses": ...    # OpenAI
        elif self.formato == "messages": ...  # Claude
        elif self.formato == "interactions": ...
        else:  # "chat": Gemini (OpenRouter), Ollama
```

max_retries=3 en los clientes: reintentos con espera exponencial.

formato elige la rama: responses, messages, interactions o chat (Chat Completions).

coste_usd es una propiedad: None si el modelo no está en PRECIOS.

El modelo sale de .env salvo que lo pases al crear el LLM.

PRÁCTICA · 20:55 · 25 MIN

P4 · Comparativa de proveedores

ENUNCIADO Y CÓDIGO · S1-LLM-APIS/	PROGRAMAS (TODO)	EJECUTA
`P4-comparativa-de-proveedores/`	`s1_comparativa.py`	`python s1_comparativa.py`

1

Completa el bucle (TODO) y ejecútalo: 5 correos × 3 proveedores → comparativa.csv con segundos, tokens y coste.

2

`calidad_1a5`

Puntuad en equipo la calidad de cada respuesta, de 1 a 5.

3

¿Qué modelo elegiríais para contestar 2.000 correos al mes, y por qué?

Con datos de la tabla (coste, latencia y calidad), no con impresiones.

PRÁCTICA OPCIONAL

P5 · Streaming y modelo local

ENUNCIADO Y CÓDIGO · S1-LLM-APIS/	SOLO EJECUTAS	EJECUTA
P5-streaming-y-modelo-local/	s1_streaming.py	python s1_streaming.py

s1_streaming.py · fragmento de OpenAI

```
for ev in cliente.responses.create(
        model=modelo, input=PREGUNTA,
        stream=True):
    if ev.type == "response.output_text.delta":
        print(ev.delta, end="", flush=True)
```

El texto aparece según se genera: menos latencia percibida, mismo coste.

Modelo local

```
ollama pull llama3.2

LLM("ollama").responder(...)
```

Pruébalo con los correos de la comparativa y compara calidad.

s1_streaming.py


```
"""Sesión 1. Streaming: mostrar la respuesta según se genera (mejora la latencia percibida)."""
import os

import anthropic
import openai
from dotenv import load_dotenv
from google import genai

load_dotenv()
PREGUNTA = "Explica en cinco líneas qué es un acuerdo de nivel de servicio (SLA)."

print("--- OpenAI")
for evento in openai.OpenAI().responses.create(
    model=os.environ["OPENAI_MODEL"], input=PREGUNTA, stream=True
):
    if evento.type == "response.output_text.delta":
        print(evento.delta, end="", flush=True)
```

s1_streaming.py


```
print("\n--- Claude")
with anthropic.Anthropic().messages.stream(
    model=os.environ["ANTHROPIC_MODEL"], max_tokens=512,
    messages=[{"role": "user", "content": PREGUNTA}],
) as flujo:
    for trozo in flujo.text_stream:
        print(trozo, end="", flush=True)

print("\n--- Gemini")
if os.getenv("GEMINI_API_KEY"): # opcional: API nativa de Gemini
    cliente = genai.Client() # en una variable: si no, el SDK cierra el cliente antes de usarlo
    for evento in cliente.interactions.create(
        model=os.environ["GEMINI_MODEL"].removeprefix("google/"), input=PREGUNTA, stream=True
    ):
        if evento.event_type == "step.delta" and evento.delta.type == "text":
            print(evento.delta.text, end="", flush=True)
else: # por defecto, vía OpenRouter con Chat Completions
    cliente = openai.OpenAI(base_url="https://openrouter.ai/api/v1",
```

s1_streaming.py


```
                            api_key=os.environ["OPENROUTER_API_KEY"])
    for trozo in cliente.chat.completions.create(
        model=os.environ["GEMINI_MODEL"], messages=[{"role": "user", "content": PREGUNTA}],
        stream=True,
    ):
        if trozo.choices and trozo.choices[0].delta.content:
            print(trozo.choices[0].delta.content, end="", flush=True)
print()
```

21:20 · CIERRE

Preguntas de control

1. Si en la segunda llamada no reenvías la primera pregunta, ¿por qué el modelo «no se acuerda»?
2. ¿Por qué cuestan más los tokens de salida y qué implica al diseñar los prompts?
3. Una compañera quiere llamar a la API de Gemini desde el JavaScript de su web. ¿Qué le recomiendas?
4. ¿Cómo debe reaccionar tu código ante un 429? ¿Y ante un 401?
5. ¿Qué gana la aplicación con el patrón Adapter?

PARA SEGUIR

Recursos

OpenAI	Responses API, modelos, precios y cambios · developers.openai.com/api/docs
Anthropic	Messages API, modelos y precios · platform.claude.com/docs
Google	Primeros pasos con la Interactions API · ai.google.dev/gemini-api/docs
OpenRouter	openrouter.ai/docs · modelos y precios: openrouter.ai/models
Ollama	Modelos abiertos en local, API compatible con OpenAI · ollama.com

ENTREGABLE E1 · 10 %

llm_client.py con tres proveedores y comparativa razonada

comparativa.csv puntuada y analisis.md: coste, latencia y calidad, y vuestra elección de modelo para 2.000 correos al mes.

Carpeta e1/ de tu repo de entregas · Viernes 16 de octubre, 23:59

Próxima sesión

Salidas estructuradas y uso de herramientas

Traed tostadero.db creada (python crear_bd.py)